

LegalEase: AI-Powered Legal Document Generator

Naan Mudhalvan Project Documentation

Sub Topic: Create and Configure main.py

This project section explains how to create and configure the main.py entry point of the LegalEase FastAPI backend. The main.py file initializes the application, configures metadata, registers routers, enables middleware, prepares documentation, and provides basic service-level configuration.

Project Goal: Build a clean and maintainable FastAPI application entry point that connects all LegalEase backend modules.

2. Introduction to main.py

In a FastAPI project, main.py commonly acts as the application entry point. It creates the FastAPI application object and connects the different components required by the backend.

For LegalEase, main.py is responsible for creating the API instance, defining project metadata, registering route modules, configuring middleware, and exposing the application to the ASGI server.

Keeping the entry point organized is important because a growing application can contain authentication routes, document routes, health checks, database dependencies, AI services and file-generation modules.

The main.py file should therefore contain application configuration and integration code rather than all business logic. Document-generation rules should remain in dedicated service modules.

This design makes LegalEase easier to test, maintain and extend.

3. Purpose of the Main Application File

The primary purpose of main.py is to construct the FastAPI application and provide a single object that can be executed by Uvicorn or another ASGI server.

```
from fastapi import FastAPI

app = FastAPI(
    title='LegalEase API',
    version='1.0.0',
    description='AI-powered legal document generation backend'
)
```

The application object becomes the central point through which HTTP requests enter the backend. FastAPI uses the route definitions registered with this object to determine how requests should be handled.

Project metadata such as title, version and description also appears in the automatically generated OpenAPI documentation. This makes the API easier for developers and evaluators to understand.

4. Creating the FastAPI Application

The first implementation step is to import FastAPI and create an application instance. The instance should have meaningful metadata for the LegalEase project.

```
from fastapi import FastAPI

app = FastAPI(
    title='LegalEase API',
    description='Backend for LegalEase AI-Powered Legal Document Generator',
    version='1.0.0',
    docs_url='/docs',
    redoc_url='/redoc'
)
```

The `docs_url` and `redoc_url` settings control the locations of the interactive API documentation pages. During development, these pages are useful for testing endpoints without requiring a separate frontend.

A version number allows the project to communicate which API release is currently running. Later, API routes can also be grouped under a versioned prefix such as `/api/v1`.

5. Configuring Application Metadata

Application metadata gives the backend a professional identity. The title identifies the system, the description explains its purpose, and the version identifies the current release.

Example metadata for LegalEase can describe the service as an AI-assisted legal-document drafting platform. The wording should clearly communicate that generated documents are drafts and may require appropriate review.

```
app = FastAPI(  
    title='LegalEase API',  
    description='REST API for AI-assisted legal document drafting',  
    version='1.0.0',  
    contact={  
        'name': 'LegalEase Project Team'  
    }  
)
```

Metadata appears in the OpenAPI schema and helps frontend developers understand the service. It also improves the presentation of the Naan Mudhalvan project during demonstration.

6. Configuring API Routers

A large FastAPI application should not place every endpoint inside `main.py`. Instead, individual routers should be created for authentication, documents, health checks and other modules.

```
from fastapi import FastAPI
from app.api.routes.documents import router as document_router
from app.api.routes.auth import router as auth_router
from app.api.routes.health import router as health_router

app = FastAPI(title='LegalEase API', version='1.0.0')

app.include_router(document_router, prefix='/api/v1')
app.include_router(auth_router, prefix='/api/v1')
app.include_router(health_router)
```

This approach keeps `main.py` short and readable. Each router can focus on a specific responsibility while the main file acts as the integration point.

The version prefix also makes it possible to introduce a future API version without immediately breaking existing clients.

7. Creating the Health Check

A health-check endpoint is useful for confirming that the FastAPI application is running. It can also be used by deployment infrastructure to monitor service availability.

```
from fastapi import APIRouter

router = APIRouter(tags=['health'])

@router.get('/health')
def health_check():
    return {
        'status': 'ok',
        'service': 'LegalEase API'
    }
```

The health router is registered in `main.py`. A successful response confirms that the API process can receive and handle requests.

More advanced deployments can add readiness checks for the database or other required services. The basic student project can begin with a lightweight application health endpoint.

8. Configuring Middleware

Middleware runs around HTTP request processing. LegalEase may require middleware for cross-origin access, request logging, security headers or other application-wide behavior.

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:3000'],
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*']
)
```

CORS configuration is important when the LegalEase frontend and backend run on different origins. In production, allowed origins should be restricted to trusted frontend domains rather than allowing every origin.

Middleware should be added carefully because incorrect settings can create security or integration problems. Environment-specific configuration is preferable to hard-coded production values.

9. Startup and Shutdown Configuration

FastAPI supports application lifecycle events. LegalEase can use these events for lightweight startup and shutdown operations such as checking configuration, initializing resources or closing resources.

```
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    print('LegalEase API starting...')
    yield
    print('LegalEase API shutting down...')

app = FastAPI(
    title='LegalEase API',
    lifespan=lifespan
)
```

The lifespan mechanism provides a centralized place for application lifecycle behavior. Heavy or unnecessary work should not be placed in startup because it can delay service availability.

For the academic project, lifecycle handling can demonstrate that the application is prepared for database connections, service initialization and graceful shutdown.

10. Exception Handling in main.py

main.py can register application-wide exception handlers. These handlers help LegalEase return consistent responses when unexpected errors occur.

```
from fastapi import Request
from fastapi.responses import JSONResponse

@app.exception_handler(Exception)
async def general_exception_handler(request: Request, exc: Exception):
    return JSONResponse(
        status_code=500,
        content={'detail': 'Internal server error'}
    )
```

Internal exception details should not be exposed to users. Developers can use server-side logging to investigate failures while the API returns a safe message.

Specific application errors can use custom exception classes and dedicated handlers. This creates predictable responses for validation, document-generation and authorization failures.

11. Configuring CORS for LegalEase

The LegalEase frontend may be developed separately from the FastAPI backend. If the frontend runs at another origin, the backend must explicitly allow the required origin through CORS settings.

```
ALLOWED_ORIGINS = [  
    'http://localhost:3000',  
    'http://localhost:5173'  
]  
  
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=ALLOWED_ORIGINS,  
    allow_credentials=True,  
    allow_methods=['GET', 'POST', 'PUT', 'DELETE'],  
    allow_headers=['*']  
)
```

Development origins can be different from production origins. The safest approach is to load allowed origins from environment configuration.

Correct CORS configuration enables the frontend to communicate with the API while avoiding unnecessarily broad cross-origin access.

12. Configuring Environment Settings

Application configuration should not be hard-coded throughout main.py. A settings module can load environment variables and provide configuration values to the application.

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    app_name: str = 'LegalEase API'
    version: str = '1.0.0'
    database_url: str
    secret_key: str
    ai_api_key: str | None = None

settings = Settings()
```

main.py can then use settings.app_name and settings.version. Secrets such as database credentials and AI keys should never be committed to source control.

An example environment file can document the required variables while the actual secret values remain private.

13. Complete main.py Example

The following example shows how the major configuration elements can be brought together in one main.py file.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.api.routes.documents import router as document_router
from app.api.routes.auth import router as auth_router
from app.api.routes.health import router as health_router

app = FastAPI(
    title='LegalEase API',
    description='AI-assisted legal document generation backend',
    version='1.0.0'
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:3000'],
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*']
)

app.include_router(document_router, prefix='/api/v1')
app.include_router(auth_router, prefix='/api/v1')
app.include_router(health_router)

@app.get('/')
def root():
    return {'message': 'Welcome to LegalEase API'}
```

This structure keeps main.py focused on application composition. Individual features remain in their respective modules.

14. Root Endpoint and API Welcome Message

A simple root endpoint can confirm that the service is available and provide a basic welcome message. It is not a replacement for the health endpoint but can improve the initial API experience.

```
@app.get('/')
def root():
    return {
        'project': 'LegalEase',
        'message': 'AI-Powered Legal Document Generator API',
        'version': '1.0.0'
    }
```

The root endpoint can also provide a link or reference to the API documentation in a production-friendly implementation.

The response should remain lightweight. Complex business operations should be handled through dedicated endpoints.

15. Connecting Database Initialization

main.py can integrate the database initialization process without containing database implementation details. A database module can expose an initialization function or lifecycle handler.

```
from app.db.database import init_db

@asynccontextmanager
async def lifespan(app: FastAPI):
    init_db()
    yield

app = FastAPI(lifespan=lifespan)
```

Database tables should normally be managed through a migration system in a larger deployment. For a student prototype, an initialization function can demonstrate the connection between the application and database.

Keeping database code in app/db prevents main.py from becoming tightly coupled to SQLAlchemy implementation details.

16. Running main.py with Uvicorn

Once main.py has been created, Uvicorn can load the app object and start the FastAPI server.

```
uvicorn app.main:app --reload
```

Here, app.main identifies the Python module and :app identifies the FastAPI application object. The --reload option is useful during development because the server automatically reloads after code changes.

The application can then be accessed through the local server address. FastAPI's interactive documentation can be opened at the configured documentation route.

```
http://127.0.0.1:8000/docs
```

For production, a suitable deployment configuration should be used instead of development reload mode.

17. Testing the Main Application

The main application should be tested to confirm that routes are registered and the configuration behaves as expected.

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_root():
    response = client.get('/')
    assert response.status_code == 200
    assert response.json()['project'] == 'LegalEase'

def test_health():
    response = client.get('/health')
    assert response.status_code == 200
```

Testing `main.py` helps identify missing routers, incorrect prefixes, invalid middleware configuration and startup problems before the application is deployed.

The test suite can later include document-generation endpoints, authentication, database behavior and file downloads.

18. Security and Production Configuration

The main application configuration should support secure production behavior. Debug settings, unrestricted CORS and development secrets should not be used in a public deployment.

Security-related configuration includes trusted origins, HTTPS at the deployment layer, secure authentication, request validation, rate limiting where appropriate, safe file handling and protection of secret values.

Logging should be configured so that failures can be investigated without exposing sensitive legal-document contents, passwords, access tokens or API keys.

The application should also use maintained dependencies and receive regular security updates. Configuration should be reviewed before every production deployment.

19. Expected Results and Project Demonstration

After completing `main.py`, the LegalEase backend should start successfully and expose the configured API routes. The application should display its project metadata through OpenAPI documentation and respond to health and root requests.

Suggested demonstration:

1. Activate the virtual environment.
2. Start Uvicorn.
3. Open the `/docs` page.
4. Test the root endpoint.
5. Test the health endpoint.
6. Demonstrate the document router.
7. Show request validation.
8. Demonstrate CORS configuration with the frontend.
9. Run the automated tests.

This demonstrates that `main.py` is not just a startup script; it is the integration point that connects the major modules of the LegalEase application.

Future improvements can include API versioning, structured logging, monitoring, dependency injection enhancements, asynchronous workflows and cloud deployment configuration.

20. Conclusion

Creating and configuring `main.py` is an important step in building the LegalEase FastAPI backend. It establishes the application instance, project metadata, router registration, middleware, lifecycle behavior, exception handling and integration points for the database and other services.

A well-organized `main.py` keeps the application entry point simple while allowing the project to grow through modular routers and services. This approach improves maintainability, testing and future development.

For the Naan Mudhalvan project, the implementation demonstrates practical knowledge of Python, FastAPI, REST architecture, application configuration, middleware, API documentation, testing and deployment preparation.

Final Project Statement: LegalEase combines a guided user interface, reusable document templates, AI-assisted drafting and a structured FastAPI backend. The properly configured `main.py` file acts as the central foundation that brings these components together into a complete application.

— End of 20-Page Project Section —